An Architectural Overview of Graph of Thoughts (GoT) in Large Language Models

by Alexandru Bratosin

Main Thought

The rapid evolution of Large Language Models (LLMs) has been heavily driven by prompt engineering techniques that elicit complex reasoning. While Chain of Thought (CoT) and Tree of Thoughts (ToT) have significantly improved model performance by structuring reasoning into linear paths and hierarchical trees, they fail to capture the non-linear, synergistic nature of human problem-solving. This paper details the **Graph of Thoughts (GoT)** framework, a paradigm that models LLM reasoning as an arbitrary graph. By allowing thought states to branch, merge, and loop, GoT enables complex state evaluation and synthesis, offering a more robust architecture for multifaceted computational tasks.

1. Introduction

Human thought is rarely a strict linear progression. When solving complex problems, we explore multiple avenues, combine successful ideas from different paths, and backtrack to refine previous assumptions. Early reasoning frameworks for LLMs, such as CoT, constrained the model to a single linear sequence. ToT introduced branching, allowing models to explore multiple paths simultaneously. However, ToT lacks a mechanism for *synergy*—the ability to merge distinct branches of thought into a single, cohesive conclusion.

Graph of Thoughts (GoT) addresses this limitation by formalizing the reasoning process as a directed graph. In this architecture, individual thoughts can interact, combine, and iterate, allowing the LLM to solve tasks that require non-linear logic, such as complex sorting, scheduling, and creative synthesis.

2. Core Architecture of GoT

The GoT framework can be mathematically formalized as a directed graph $G = (V, E)$.

- **Vertices (V):** Each vertex $v \in V$ represents a distinct "thought" or an intermediate state of reasoning. A thought can be a paragraph, a block of code, or a mathematical formula, depending on the task domain.
- **Edges (E):** Each edge $e = (v_i, v_j)$ represents a dependency. If an edge exists from v_i to v_j, it dictates that the generation of thought v_j used v_i as part of its prompt context.

2.1 Thought Transformations

GoT operates by applying specific transformations to the vertices in the graph. The primary operations include:

1. **Generation (Branching):** Generating one or multiple new thoughts from a single existing thought.
2. **Aggregation (Synergy):** Taking multiple distinct thoughts and merging them into a unified, superior thought. This is the defining feature that ToT lacks.

3. **Refinement (Looping):** Iteratively improving a single thought by passing it back through the LLM with a critique prompt.

2.2 Scoring and State Selection

To prevent exponential graph growth, GoT employs a scoring function $S(v)$ to evaluate the quality of each thought. The system actively prunes low-scoring vertices and retains only the most promising candidates for subsequent transformations.

3. Implementation Design

To operationalize GoT, we require a framework capable of managing state, orchestrating LLM calls, and dynamically updating the graph structure. Below is a conceptual Python implementation utilizing a lightweight, object-oriented approach.

3.1 Defining the Thought Node

First, we define a structure to hold individual thoughts and their metadata, including their historical lineage (edges) and their assigned scores.

Python

```
import uuid
from typing import List, Optional

class ThoughtNode:
    """Represents a single vertex in the Graph of Thoughts."""
    def __init__(self, content: str, parents: Optional[List['ThoughtNode']] = None):
        self.id = str(uuid.uuid4())
        self.content = content
        self.parents = parents or []
        self.score: float = 0.0

    def add_parent(self, parent_node: 'ThoughtNode'):
        self.parents.append(parent_node)

    def __repr__(self):
        return f"<Thought {self.id[:6]} | Score: {self.score}>"
```

3.2 Graph Execution and Synergy

The core advantage of GoT is the synergize operation. The executor evaluates multiple independent paths and prompts the LLM to merge them, extracting the best elements from each.

Python

```
class GoTExecutor:
    """Orchestrates the graph transformations."""
    def __init__(self, llm_client):
        self.llm = llm_client
        self.graph: List[ThoughtNode] = []
```

```
    def evaluate(self, node: ThoughtNode) -> float:
        """Prompts the LLM to score the thought out of 10."""
            prompt = f"Rate the logical coherence of this thought on a scale of 0-10: {node.content}"
        score = float(self.llm.generate(prompt)) # Abstracted LLM call
        node.score = score
        return score

    def synergize(self, nodes: List[ThoughtNode]) -> ThoughtNode:
        """Merges multiple thought nodes into a single, cohesive node."""
        combined_content = "\n".join([f"- {n.content}" for n in nodes])
                prompt = f"Combine the following ideas into a single, optimized solution:\n{combined_content}"

        # Abstracted LLM generation
        merged_output = self.llm.generate(prompt)

        # Create new node representing the merged state
        merged_node = ThoughtNode(content=merged_output, parents=nodes)
        self.graph.append(merged_node)

        return merged_node
```

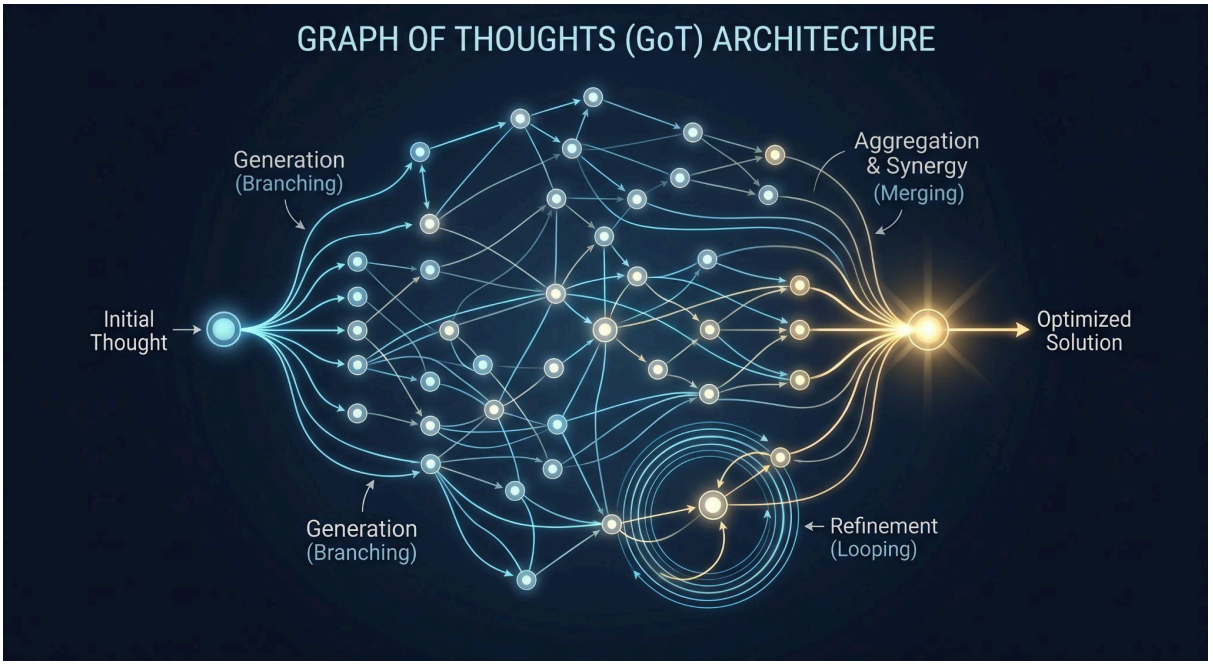
In practice, a main execution loop would iteratively generate nodes, score them, retain the top k nodes, and synergize them before progressing to the next stage of the problem.

4. Architectural Advantages

1. **Context Efficiency:** By extracting and merging only the successful components of different thought branches, GoT reduces the accumulation of redundant or erroneous tokens in the context window.
2. **Cross-Pollination of Ideas:** In tasks requiring creative synthesis (e.g., architectural design or complex software engineering), early conceptual branches often contain distinct, valuable insights. GoT's graph structure ensures these insights can be unified rather than existing in isolated silos.
3. **Self-Correction:** The presence of cyclic dependencies (loops) allows the model to act as its own critic, refining an answer iteratively until a predefined threshold $S(v) > \tau$ is met.

5. Conclusion

Graph of Thoughts represents a necessary evolution in prompt engineering and LLM orchestration. By moving away from rigid, linear constraints and adopting a dynamic network of interacting thoughts, GoT aligns machine reasoning more closely with human cognitive processes. Future implementations will likely see the integration of specialized, smaller language models handling the scoring and routing tasks, reserving the primary heavy-parameter LLM exclusively for high-value node generation and synergy.


GRAPH OF THOUGHTS (GoT) ARCHITECTURE
Generation
(Branching)
Aggregation
& Synergy
(Merging)
Initial
Thought
Optimized
Solution
Generation
(Branching)
Refinement
(Looping)